

FAQ: Sharding with MongoDB

This document answers common questions about Sharding. See also the Sharding section in the manual, which provides an overview of sharding, including details on:

- [Shard Keys and Considerations for Shard Key Selection](#)
- [Query Routing](#)
- [High Availability](#)
- [Data Partitioning with Chunks and Chunk Migration Process](#)
- [Troubleshoot Sharded Clusters](#)

Is sharding appropriate for a new deployment?

Sometimes. However, if your data set fits on a single server, you should begin with an unsharded deployment as sharding while your data set is small provides *little advantage* .

Can I select a different shard key after sharding a collection?

Your options for changing a shard key depend on the version of MongoDB that you are running:

- Starting in MongoDB 5.0, you can reshard a collection by changing a document's shard key.
- You can refine a shard key by adding a suffix field or fields to the existing shard key.

TIP

See also:

[Shard Keys](#)

Why are my documents not distributed across the shards?

The balancer starts distributing data across the shards once the distribution of chunks has reached certain thresholds. See [Migration Thresholds](#).

In addition, MongoDB cannot move a chunk if the number of documents in the chunk exceeds a certain number. See [Maximum Number of Documents Per Range to Migrate and Indivisible/Jumbo Chunks](#).

How does `mongos` detect changes in the sharded cluster configuration?

`mongos` instances maintain a cache of the config database that holds the metadata for the sharded cluster.

`mongos` updates its cache lazily by issuing a request to a shard and discovering that its metadata is out of date. To force the `mongos` to reload its cache, you can run the `flushRouterConfig` command against each `mongos` directly.

How does `mongos` use connections?

Each `mongos` instance maintains a pool of connections to the members of the sharded cluster. Client requests use these connections one at a time; i.e. requests are not multiplexed or pipelined.

When client requests complete, the `mongos` returns the connection to the pool. These pools do not shrink when the number of clients decreases. This can lead to an unused `mongos` with a large number of open connections. If the `mongos` is no longer in use, it is safe to restart the process to close existing connections.

To return aggregated statistics related to all of the outgoing connection pools used by the `mongos`, connect `mongosh` to the `mongos`, and run the `connPoolStats` command:

```
db.adminCommand("connPoolStats");
```



See the System Resource Utilization section of the UNIX `ulimit` Settings for Self-Managed Deployments document.

